

Rapport Langage Go – 29/05/2026

Cauvet Louis, M2 IW à l'ESGI

Rappel sur les types en Go

- Il n'y a pas de conversion implicite en Go.
- Lorsqu'on déclare une variable par défaut, sa valeur est nulle.
- Un slice est un tableau dont la taille est modifiable en cours d'usage.
- Go est capable de détecter lui-même le type d'une variable lors de sa déclaration, grâce à l'inférence de type
- On peut déclarer une variable dans sa version courte ("score := 42"), mais uniquement dans les fonctions, car le "!=" n'est valide que dans le corps des fonctions
- On peut déclarer plusieurs variables sur la même ligne ("x,y := 10, 20")
- Go peut échanger des valeurs très simplement (x, y = y, x)
- Le compilateur Go refuse le code qui déclare des variables non utilisées
- Le iota s'incrémente à chaque constante dans un groupe const()

Surcharge de fonctions

Dans le langage Go, il n'existe pas de surcharge de fonction : on ne peut pas faire des déclinaisons de fonctions à partir d'une déclaration de fonction initiale.

En effet, cela présente des avantages comme l'uniformisation du code mais aussi le désavantage d'embrouiller le développeur sur le long terme.

Voir exo 1 du repo git

Gestion des erreurs

Une fonction Go peut effectuer un retour multiple, pour renvoyer valeurs et erreurs à la fois grâce à un pattern idiomatique ("return a/b, nil") → Retourne par défaut "nil" si pas d'erreur/

On peut aussi ignorer une valeur de retour grâce à un _, si on est sûr que le résultat ne retournera pas d'erreur ("return a, _").

L'approche Go est plus explicite que les exceptions, car il n'y a pas de flux cachés : toute la gestion des erreurs est visible depuis le code.

Les fonctions variadiques et closures

Une fonction variadique est une fonction qui peut prendre un nombre d'arguments indéfinis pour son dernier paramètre

En Go, le terme "closure" désigne une fonction qui se souvient de l'environnement dans lequel elle a été créé. Une fonction retournant une fonction est une closure.

C'est utile dans un souci d'optimisation des ressources, comme par exemple dans des middleware http, des tris personnalisés ou des callbacks.

Voir exo 2 du répo git

Les fallthrough

En Go, le terme "fallthrough" s'utilise dans un switch/case, afin d'exécuter toutes les étapes qui suivent l'étape qui possède le fallthrough.

Cette notion n'a pas les mêmes propriétés de la même manière dans les différents langages, et n'existe même pas en Python ou en Rust par exemple.

Ainsi, dans Go, le fallthrough permet d'éviter les erreurs classiques de l'utilisation d'un switch/case

Voir exo 3 du répo git

Les Maps

En Go, la notion de "Map" est similaire à un dictionnaire clé/valeur

On peut créer une map avec la fonction "make()" qui initialise une map prête à l'emploi.

Ex : `m := make(map[string]int)`

Les structures

D'un point de vue objet, une structure correspond à un élément qui possède une ou plusieurs propriétés. Ces propriétés peuvent être de tout type, notamment des slices.

Il y a 3 manières d'initialiser une structure :

- Façon positionnelle :

- Façon nommée : On déclare une structure au même format que la déclaration initiale de la structure. C'est celle qui se rapproche le plus de la programmation Orientée Objet
- Façon avec une valeur zéro puis une affectation, qui est plus solide.

On peut travailler sur les structures de différentes manières :

- Soit on travaille sur une copie sans modifier l'originale (méthode "value receiver")
- Soit on travaille directement sur le vrai objet (méthode "pointer receiver")

On récupère l'adresse mémoire d'un pointeur avec "&" (ex : "&employees[0]"), et on accède à la valeur qu'il pointe avec "*" (ex : "(*premier).poste")

Les tags

Les tags sont des métadonnées pour Json.

Ils sont déclarés sous forme de chaîne entre backsticks.

Attention : certaines données sensibles telles que les mots de passe ne doivent pas être renseignées dans les tags

Une fois déclarées, on peut ensuite encoder ces données en JSON avec la fonction "json.marshal()"

Voir exo 4 du répo git

La visibilité

Dans le langage Go, la visibilité dépend uniquement de casse de la première lettre d'un identifiant. On ne retrouve donc pas les mots-clés habituels comme "public", "private"...

La sémantique est donc très importante :

Une fonction avec une majuscule est exportable (= public) et accessible depuis n'importe quel package, alors qu'une fonction en minuscule ne sera pas exportable (= privée) et sera utilisée dans le package courant uniquement.

Attention : Go s'appuie sur le camelCase, donc par exemple "maFonction" ne sera pas exportable mais "MaFonction" le sera

Cette règle s'applique à tous les identifiants (types, fonctions, variables, champs de structures...).

Cependant, même si une structure est exportée, ses champs en minuscule restent inaccessibles de l'extérieur

Garantir l'exécution d'une fonction

En Go, le mot-clé "defer" permet de systématiquement exécuter du code dans une fonction avant le "return", même si une erreur est rencontrée dans une exécution préalable.

S'il y a plusieurs defer dans la fonction, le dernier defer (celui situé le plus bas) sera le premier exécuté.

Packages standards

Parmi les packages standards de Go :

- "fmt" est destiné au formatage et à l'affichage
- "strings" est destiné à la manipulation de chaînes ("Contains", "ToUpper", "Split"...)
- "strconv" est destiné à convertir des chaînes
- "math" est destiné aux fonctions mathématiques
- "sort" est destiné aux tris